MathDevMCP

Final Release Report for an Industrial Mathematical Development Agent

Chak Wong and collaborators

April 29, 2026

Contents

Part I

Release Claim

Chapter 1

Executive Summary

Mathematical teams already know the failure modes that matter here. A note says a likelihood uses a log determinant and a linear solve, while the implementation has been refactored twice under deadline. A proof sketch depends on positive-definiteness or invertibility, but the assumption is buried in another section or carried only in a senior colleague's memory. A new agent can sound confident about the mathematics, but the answer is hard to trust because it does not show where the claim came from or where the verification boundary ends.

MathDevMCP is built for those moments. It gives colleagues a local-first way to connect claims to the implementation, surface support and mismatches, and avoid making unsupported mathematical statements. Instead of asking a reviewer to trust an agent's paraphrase, it turns local LaTeX documents, labels, code, parser signals, typed obligations, backend checks, and release gates into structured evidence packets that a human can inspect, challenge, and reuse.

The product promise is practical rather than grand. MathDevMCP helps a colleague find the right equation and its nearby assumptions, detect that code no longer carries a documented solve-like operation, explain why a derivation remains uncertified, and hand another reviewer a document-grounded implementation brief. It is most useful in review loops where unsupported claims are expensive and where a generic assistant is too willing to smooth over missing provenance.

This is why the release claim is deliberately narrow and evidence based. MathDevMCP does not claim to prove arbitrary mathematical papers correct. Its value is that it preserves the distinction between retrieval, diagnostics, mismatch, abstention, and deterministic certification in machine-readable reports. In practice, that means colleagues can move faster without pretending that a search hit, a parser extraction, or a nearby symbolic pattern is already a proof.

The current full release profile is configured to require benchmark evidence, parser policy, governance validation, release-corpus metadata, LeanDojo backend environment evidence, LaTeXML validation, and a private-corpus manifest. The private-corpus evidence

used for this report is an external sanitized corpus generated outside the repository. Its purpose is to validate the privacy and release-gate machinery without committing private department documents. A real department manifest can be substituted through the same environment variable.

Release status. With an external sanitized private manifest supplied, the full profile has no blocking findings. Generated report snippets may record a dirty working tree while the report is being edited; the post-commit release readiness gate should be rerun before a release tag is cut.

1.1 Why This Tool Exists

MathDevMCP exists because mathematical review work breaks down in predictable ways. Derivations drift from code, hidden assumptions disappear into nearby sections, and ordinary assistants can retrieve plausible context without showing whether the implementation carries the required operation or whether the derivation remains uncertified. The tool is useful precisely because it keeps labels, file paths, line spans, code comparisons, typed diagnostics, and certification boundaries visible. It does not replace expert judgment; it gives expert judgment a better starting point.

Chapter 2

MCP Conversations That Sell The Tool

The most natural way to use MathDevMCP is not as a long list of commands. It is as a conversation with an agent that has a small set of disciplined tools. The colleague asks the same question they would ask in a hallway or code review: "Does this implementation still match the equation in the note?" The difference is that the agent can answer by calling bounded MCP tools, citing labels and files, and preserving the verification boundary.

This chapter is intentionally early in the report because it is the adoption story. A colleague does not need to learn every command before seeing value. They need to learn the shape of a good prompt, the kind of tool call it triggers, and the kind of answer that should come back.

2.1 The Conversation Pattern

A productive MCP conversation usually has five turns:

1. ask the agent to locate the mathematical claim in a local root;
2. ask for the surrounding assumptions or notation;
3. ask the agent to compare the claim with a concrete code file;
4. ask whether the remaining derivation is certified, diagnostic, or unverified;
5. ask for a short handoff note that another reviewer can reproduce.

The important behavior is not that the agent sounds confident. The important behavior is that it chooses tools such as `search_latex`, `extract_latex_neighborhood`, `compare_label_code`, `audit_derivation_v2_label`, and `implementation_brief` before making a mathematical claim.

2.2 A Realistic Review Transcript

Listing 2.1: MCP conversation for a document-code review

```
User:
  In /path/to/project, check whether the Kalman likelihood in the notes
  still matches src/kalman_filter.py. I care about logdet, solve, and the
  quadratic form. Keep the proof boundary explicit.

Agent:
  I will first find the likely label, then compare that label to the code.

Tool: search_latex
  root=/path/to/project
  query="Kalman likelihood log determinant solve"
  limit=3

Tool result:
  label=eq:dept-state-space-likelihood
  file=doc_department_state_space.tex
  lines=...

Agent:
  I found label eq:dept-state-space-likelihood. I will inspect nearby context
  before comparing the implementation.

Tool: extract_latex_neighborhood
  root=/path/to/project
  label=eq:dept-state-space-likelihood

Tool result:
  status=found
  context includes the likelihood block and nearby state-space assumptions

Tool: compare_label_code
  root=/path/to/project
  label=eq:dept-state-space-likelihood
  code=/path/to/project/src/kalman_filter.py
  required_terms=["logdet", "solve", "quadratic_form"]
  paragraph_context=true

Tool result:
  status=mismatch
  matched_terms=["logdet"]
  missing_terms=["solve", "quadratic_form"]

Agent:
  The checked code path visibly carries logdet, but the configured review
  terms solve and quadratic_form are missing. This is not a theorem that the
  code is wrong. It is a document-grounded drift signal. The next review step
  is to inspect whether the quadratic term is implemented through an equivalent
  factorization or whether the solve-like operation was dropped.

User:
  Can you also tell me whether the derivation itself is certified?

Tool: audit_derivation_v2_label
  root=/path/to/project
  label=eq:dept-state-space-likelihood
```

```
  summary_only=true

Tool result:
  status=unverified
  high_priority_actions=[
    "state or verify invertibility",
    "check log-det domain",
    "check conformability",
    "check linear-solve residual"
  ]

Agent:
  The derivation audit is diagnostic, not a certificate. I would write the
  review note as: the label was found, the implementation comparison reports
  a missing solve/quadratic-form signal, and the derivation remains unverified
  until the matrix assumptions and solve residual are reviewed.
```

2.3 Prompt Patterns That Trigger Tools

Prompt shape	MCP tool	Why it helps
"Find the equation or assumption for ..."	`search_latex`	Returns labels and provenance instead of memory.
"Show the context around label ..."	`extract_latex_neighborhood`	Surfaces nearby assumptions and notation.
"Compare this label to this code file ..."	`compare_label_code`	Turns a vague drift worry into matched and missing terms.
"Check whether this derivation is certified."	`audit_derivation_v2_label`	Keeps the distinction between proof, diagnostic evidence, and abstention.
"Give me a handoff note for review."	`implementation_brief`	Bundles search, context, comparison, and next action.
"Is this release ready?"	`release_readiness`	Reports blockers and caveats instead of informal readiness claims.

2.4 When MCP Tools Trigger Instead of LLM-Only Reasoning

A useful mental model is that the agent has two modes. In an LLM-only answer, the agent reasons from the prompt, the chat history, and any text the user has pasted into the conversation. That can be appropriate for ordinary explanation, brainstorming, rewriting, or conceptual discussion. In an MCP-grounded answer, the agent asks MathDevMCP for bounded evidence from the repository: labels, file paths, line spans, code comparisons, derivation diagnostics, benchmark results, or release-readiness reports.

The difference matters because MathDevMCP does not automatically run on every mathematical word in a conversation. A client model still decides whether to call a tool. The user can make the decision almost unavoidable by asking for local evidence, naming a repository root, naming a label or code path, or asking for a status that only a tool should produce. If the response contains no tool name, no arguments, no label provenance, and no result fields, the user should treat it as an LLM-only answer.

Situation	LLM-only answer is reasonable	MCP-grounded answer is expected
Conceptual explanation	"Explain why Kalman likelihoods contain a log determinant." The answer can explain the mathematics without inspecting local files.	"In this repository, find the likelihood label and cite the file and line span." This asks for local provenance, so `search_latex` should run.
Editing prose	"Make this paragraph clearer." The user supplies the text and does not ask for repository evidence.	"Rewrite this paragraph after checking the nearby theorem assumptions." The agent should retrieve context with `extract_latex_neighborhood`.
Code review	"Does this function look idiomatic?" This is a general programming judgment.	"Compare label `eq:dept-state-space-likelihood` with `src/kalman_filter.py` for `logdet` and `solve`." The agent should call `compare_label_code`.

Derivation review	"Is this derivation convincing?" Without a label or root, the agent may only reason from pasted text.	"Audit label `eq:dept-state-space-likelihood` and tell me whether the result is certified, diagnostic, or unverified." This points to `audit_derivation_v2_label`.
Release handoff	"Are we ready to release?" This is ambiguous unless the repository and profile are identified.	"Run release readiness for this repository under the full profile and list blockers and caveats." This should trigger `release_readiness`, and often `benchmark_gate` or `doctor`.

Signals That Should Cause Tool Use

The following prompt features are strong trigger signals. They do not guarantee tool use in every client, but they tell both the agent and the reviewer that an LLM-only answer would be incomplete.

1. The prompt names a local repository root, project directory, manifest, LaTeX file, or code file.

2. The prompt asks the agent to find, cite, or inspect a label, theorem, assumption, equation, section, or paragraph in local documents.

3. The prompt asks whether a document claim still matches a code path.

4. The prompt gives required mathematical or implementation terms such as `logdet`, `solve`, `gradient`, `hamiltonian`, or `euler_residual`.

5. The prompt asks for a status such as `consistent`, `mismatch`, `unverified`, `blocked`, or `release_ready`.

6. The prompt asks for derivation certification, typed obligations, backend evidence, abstentions, or proof boundaries.

7. The prompt asks for release evidence, benchmark gates, private corpus validation, governance policy, backend availability, or profile readiness.

8. The prompt says "use MCP", "show the tool result", "cite labels and files", "do not answer from memory", or "keep the evidence boundary explicit".

Cases That Look Like Tool Triggers But Usually Are Not

Many plausible prompts sound technical but still invite an LLM-only response. These are useful failure cases to show colleagues because they explain why prompt shape matters. The improved prompt does not need to be verbose; it only needs to specify the local artifact and the evidence expected.

Prompt that may not trigger MCP	Why it may stay LLM-only	Better prompt
"Explain the Kalman likelihood."	This asks for general mathematical exposition, not local evidence.	"In `/path/to/project`, find the Kalman likelihood label and summarize it with file and line provenance."
"Does the filter implementation look right?"	There is no code path, document label, or required term list.	"Compare `eq:dept-state-space-likelihood` with `src/kalman_filter.py`; check `logdet`, `solve`, and the quadratic form."
"Can MathDevMCP prove this derivation?"	The product audits bounded obligations; it does not promise unrestricted proof repair.	"Run `audit_derivation_v2_label` on this label and report whether the result is certified, diagnostic, unverified, or blocked."
"The paper and code might be inconsistent. Thoughts?"	The agent can speculate because no local target is supplied.	"Search the LaTeX for the likelihood label, then compare the best label to `src/likelihood.py` with required terms `logdet` and `solve`."
"Check the private corpus."	The tool needs a manifest path or configured environment variable, and the answer must respect redaction policy.	"Validate this private manifest with MathDevMCP, do not print private paths, and report release-gate blockers only."

"Find recent papers about Hamiltonian Monte Carlo."	That is a literature or web task, not a local MathDevMCP repository task.	"In this repository, find the HMC leapfrog label and compare it to the sampler implementation for `gradient` and `hamiltonian`."
"Summarize the equation I pasted below."	The user supplied the whole object, so the agent may summarize directly.	"After summarizing the pasted equation, search the repository for matching labels and cite any local match before making implementation claims."
"Run the release check."	The release tool requires a root and, for full evidence, a profile and optional private manifest configuration.	"Run `release_readiness` for `/path/to/project` with profile `full`, then list blockers, caveats, and missing backend evidence."

How A Colleague Can Verify That The Tool Actually Ran

The safest adoption practice is to ask the agent to expose the evidence trail. A tool-grounded answer should show at least the tool name, important arguments, and result fields. For example, a comparison answer should mention `compare_label_code`, the label, the code path, matched terms, missing terms, and the status. A derivation answer should mention `audit_derivation_v2_label`, the parser route, obligations, backend evidence or abstentions, and high-priority review actions. A release answer should mention `release_readiness`, the profile, blockers, caveats, and capability evidence.

If those artifacts are absent, the answer may still be useful, but it should be treated as explanation rather than repository evidence. The most reliable follow-up is direct: "Please rerun this using MathDevMCP tools, show the tool calls and result statuses, and do not make claims that are not supported by the tool output." That instruction converts an ambiguous prompt into an auditable MCP workflow.

2.5 Example Prompts and Expected Tool Calls

This is the colleague-facing cheat sheet for proposal and adoption discussions. The user does not need to name tools directly. They should describe the research or review problem in ordinary language, and the agent should translate that prompt into one or more narrow MCP calls before answering. If the agent answers without first grounding the request in

labels, file paths, or release evidence, the colleague should ask it to rerun the task using MathDevMCP.

Example prompt to the agent	Tools that should trigger	Evidence the answer should cite
"Find where the model note defines the likelihood used by this filter."	`search_latex`	Candidate labels, source files, line spans, and short snippets rather than a free-memory summary.
"Show me the assumptions around `eq:dept-state-space-likelihood` before you judge the implementation."	`extract_latex_neighborhood`	The labeled block plus nearby assumptions about dimensions, invertibility, conditioning, and notation.
"Compare the likelihood equation with `src/kalman_filter.py`; I care about `logdet`, `solve`, and the quadratic form."	`compare_label_code`	A status such as `consistent`, `mismatch`, or `unverified`, with matched terms, missing terms, label provenance, and code provenance.
"Can you check whether this derivation is actually certified, or only diagnostic?"	`audit_derivation_v2_label`	Parser route, typed obligations, backend evidence when available, abstentions, and high-priority human review actions.
"Before I edit the code, give me a compact implementation brief for this label and file."	`implementation_brief`	Search result, extracted context, comparison result, caveats, and a short next action that another developer can reproduce.
"Run the release-facing checks for this repository and tell me what blocks a handoff."	`benchmark_gate`, `doctor`, `release_readiness`	Benchmark totals, expected abstentions, backend availability, release profile, blockers, and caveats.
"Validate this private corpus manifest, but do not print private paths."	`validate_release_corpus`, `release_readiness`	Redacted corpus metadata, gate categories, blocked private paths, and the profile-specific readiness result.

The expected user habit is simple: ask for the mathematical task in natural language, insist on the evidence packet, and treat the agent's prose as a summary of tool output rather than as an independent authority. That habit is what turns MathDevMCP from a command collection into a practical colleague assistant.

The rest of the report shows this same pattern across the public case studies. Each case gives the mathematical situation, the generated evidence, and a short MCP-style transcript that a colleague can adapt directly in their own client.

Chapter 3

Product Scope

MathDevMCP supports five colleague-facing activities:

1. locating mathematical context in LaTeX source trees;
2. extracting labeled equation, theorem, assumption, and paragraph neighborhoods with provenance;
3. comparing document claims with Python implementations;
4. auditing derivation obligations with parser, typed, shape, numeric, and backend evidence;
5. producing repeatable release evidence across base, backend, LaTeXML, private-corpus, and full profiles.

In colleague terms, this means the tool helps answer five recurring workflow questions:

a. Where is the equation or assumption we are talking about?

b. What nearby notation or assumptions does it depend on?

c. Does the code visibly carry the documented structure?

d. If a derivation is not certified, why not?

e. Can another reviewer reproduce the evidence without trusting my memory?

The product is intentionally local-first. Source documents and code remain on the user's machine. Optional tools such as LaTeXML, Lean, LeanDojo, Sage, and SymPy are detected at runtime and reported through structured diagnostics. LeanDojo remains in the isolated `mathdevmcp-backends` environment so its dependency stack does not destabilize the main working environment.

3.1 What is included

The release includes:

- a LaTeX indexer with label, environment, and context extraction;
- code and document search utilities;
- code–document consistency checks;
- derivation-step checks with explicit *unverified* and *mismatch* outcomes;
- proof obligation and proof-audit workflows;
- typed and dimensional diagnostic workflows;
- AST operation extraction for mathematical implementation patterns;
- parser benchmark and parser policy selection;
- release corpus manifest validation;
- private-corpus validation with redacted output;
- release-readiness profiles;
- MCP facade and FastMCP server wrappers;
- operator, deployment, release, security, and maintainer documentation.

3.2 What is not claimed

MathDevMCP does not replace expert mathematical review. A parser hit is not a proof. A shape diagnostic is not a theorem. A Lean skeleton is not a certificate until direct Lean checking accepts it without placeholders. A numeric diagnostic can refute a false claim or suggest a failure mode, but it does not establish a general theorem. This boundary is a product feature: it lets colleagues move faster without laundering a plausible hint into a mathematical certificate.

3.3 Review Workflow

A normal assistant workflow tends to move from question to answer too quickly. MathDevMCP changes the order of operations: find the labeled source context, inspect nearby assumptions, compare the documented claim against code, audit the certification boundary, and hand off a compact evidence packet. That ordering is the main usability story of the tool because it makes another reviewer's local reproduction path explicit.

Chapter 4

Release Profiles

The release profiles are:

base Public benchmarks, parser policy, governance, release corpus, and normal doctor evidence.

backend Base evidence plus isolated LeanDojo backend environment validation.

latexml Base evidence plus strict LaTeXML parser validation.

private-corpus Base evidence plus an external private or sanitized private manifest.

full All of the above.

public Product-surface release evidence: CI workflow presence, packaging metadata, MCP registry and README consistency, support-matrix coverage, documentation boundary language, quality-gate execution, and generated-evidence redaction.

The profile split is important because colleagues may use the base product without every optional backend, while release managers can demand strict evidence before a broader deployment. Optional evidence is not silently ignored: strict profiles turn missing evidence into blocking findings.

The `full` profile is deliberately an internal/deployment profile. It means that optional internal evidence sources are present; it is not by itself a public industrial release claim. The `public` profile adds the product surface checks needed before a public release statement: MCP documentation must match the registry and FastMCP wrappers, the support matrix must cover every release profile, CI must run the local gates, and default public error envelopes must be structured and free of raw private paths.

4.1 Full profile evidence

Listing 4.1: Full profile release-readiness summary

```
Command: PYTHONPATH=src python -m mathdevmcp.cli release-readiness --
   root "$PWD" --profile full
Status: ready_with_caveats
Reason: Release gates passed with documented caveats.
Blockers: []
Caveats: ['dirty_worktree']
Git commit: 35b8bf7
Dirty worktree: True
```

4.2 Doctor evidence

Listing 4.2: Doctor capability summary

```
Command: PYTHONPATH=src python -m mathdevmcp.cli doctor
Overall ok: True
Python: 3.11.15 at /home/chakwong/miniconda3/envs/tfgpu/bin/python
LaTeXML: available (latexml (LaTeXML version 0.8.6))
Pandoc: available (pandoc 2.9.2.1)
Lean: available (Lean (version 4.20.0, x86_64-unknown-linux-gnu, commit
    77cfc4d1a4f6, Release))
Sage: available (SageMath version 9.5, Release Date: 2022-01-30)
LeanDojo in active env: unavailable (Python module lean_dojo is not
   importable)
SymPy: available (1.14.0)
Conflicts: 0
```

Part II

Architecture

Chapter 5

System Architecture

MathDevMCP has three layers. The first layer is a Python implementation substrate that owns parsing, indexing, consistency checks, proof-audit routing, benchmarking, release policy, and diagnostics. The second layer is a CLI and MCP facade that presents stable tools to users and agents. The third layer is a workflow and documentation layer that explains how to combine tools for colleague tasks.

The architecture is intentionally plain. The project does not hide release logic inside a server process or a notebook. A release manager can run the same Python library functions through the CLI, through MCP wrappers, and through tests. This is why the report can include command output instead of screenshots or manual claims: the system is designed around reproducible JSON contracts.

5.1 Implementation substrate

The Python package is organized around small report-producing functions. Each public report includes a metadata contract so that CLI, MCP, tests, and release docs can depend on stable shapes. For example, release readiness returns a `release_readiness_report`; parser benchmark returns `parser_backend_result`; private corpus validation returns `private_corpus_validation_report`. This is the main compatibility discipline for future maintainers.

The ownership map is practical: `release_policy.py` decides blockers and caveats; `release_corpus.py` owns public/private manifest assembly and path redaction; `parser_policy.py` chooses whether parser output is strong enough for proof-audit routing; `proof_audit_v2.py` carries the verification-boundary language; `mcp_facade.py` and `cli.py` expose the same library behavior to agents and humans. A maintainer changing a release claim should know which one of those files owns the claim before editing the report.

5.2 CLI and MCP facades

The CLI is the reproducible interface for humans and CI. The MCP facade wraps the same library functions so an agent can call them interactively. The facade does not grant unrestricted shell access. It exposes bounded tools such as `search_latex`, `compare_label_code`, `audit_derivation_v2_label`, `benchmark_gate`, and `release_readiness`.

5.3 Backend isolation

Optional backends are treated as evidence engines. LaTeXML, Pandoc, Lean, Sage, SymPy, and LeanDojo are detected by `doctor`. LeanDojo is checked through the backend environment when configured. This avoids dependency conflicts in the main Python environment while still allowing proof-search experiments and Lean fixture validation.

This split matters operationally. Colleagues can use search, context extraction, consistency checks, and release-corpus validation in the main environment. A stricter release can then demand LaTeXML and LeanDojo evidence without forcing everyday users to import every optional backend. The release profiles make that distinction visible rather than relying on informal environment knowledge.

Chapter 6

Core Data Contracts

Every user-facing workflow should be understood as a contract. A contract has a schema version, a name, a status or result payload, and where appropriate provenance. This is why downstream documentation, agents, and tests can interpret outputs without scraping prose.

The contract discipline is also what makes MathDevMCP maintainable. The CLI can pretty-print JSON, the MCP facade can return the same dictionary to an agent, and the release report can include a shortened evidence file without each layer inventing its own vocabulary. When a colleague sees `mismatch` in a brief, that word means the same thing as it means in the benchmark gate and in the tests.

6.1 Status vocabulary

The primary statuses are:

consistent The requested terms or structure were found in the checked context.

equivalent A scoped derivation step matched exactly after normalization or backend checking.

unverified Evidence exists, but it is not enough for certification.

mismatch Required structure is missing or contradicted.

inconclusive The tool lacks enough structure, parser support, or backend evidence to decide.

This vocabulary is central to the product. It prevents the agent from turning a nearby equation or diagnostic hint into a categorical mathematical claim.

Chapter 7

Parser Policy

MathDevMCP measures parser behavior before relying on parser output for proof audit. The current lightweight parser is selected for proof audit when it preserves expected labels and line provenance. LaTeXML is also validated in the strict LaTeXML profile, but current release routing prefers the provenance and environment structure available through the current parser.

The policy is deliberately conservative. Parser output becomes dangerous when an agent treats a nearby equation as if it were the exact source of a claim. The parser benchmark therefore checks label preservation, environment recognition, align-like block handling, source spans, include handling, macros, and duplicate labels. The parser selected for proof-audit routing must preserve the labels the benchmark expects and expose provenance that a colleague can inspect.

LaTeXML remains valuable because it exercises an independent parser backend and is closer to a full LaTeX processing stack. In this release it is strict profile evidence rather than the default proof-audit parser, because the lightweight parser currently gives the line-oriented provenance and environment structure used by the downstream audit tools. A future release can change that policy, but only after benchmark evidence and report examples show that the backend supports the same workflow.

The owner for this decision is `parser_policy.py`, with benchmark measurements in `parser_benchmark.py`. A maintainer should change those modules before editing this narrative, because the report is downstream of the policy evidence.

Listing 7.1: Parser policy summary

```
Command: PYTHONPATH=src python -m mathdevmcp.cli parser-benchmark --
    root "$PWD/benchmarks/fixtures" --backend current
Policy status: selected_for_proof_audit
Selected backend: current
Labels found: 67
Environments found: 67
Align-like blocks found: 4
```

```
Provenance quality: line
Blocking findings: 0
```

Chapter 8

Benchmark Gate

The benchmark gate is the main regression control. It covers consistency, derivation, workflow, proof-audit, parser-corpus, AST-corpus, typed IR, industrial review, and release-corpus cases. The gate currently requires every benchmark case to pass; expected abstentions are counted as quality signals, not as failures.

The gate is not just a CI badge. It is the release contract that stops the product from drifting into confident but unsupported behavior. Some benchmark cases are expected to return *mismatch* or *unverified*; those cases pass when the tool refuses to overclaim. This is why the report keeps expected abstentions visible. For a mathematical agent, abstention quality is a product feature: it tells colleagues that the tool can expose a gap without laundering the gap into a certificate.

When a new domain is added, the benchmark gate should grow before the marketing claim grows. The new domain needs at least one parser preservation case, one operation or code-document consistency case, and one false-confidence or abstention case if the domain contains assumptions that are easy to miss.

This makes the gate a product-management tool as well as an engineering tool: it tells the team which claims are backed by repeatable evidence and which claims still belong in the roadmap.

Listing 8.1: Benchmark gate summary

```
Command: PYTHONPATH=src python -m mathdevmcp.cli benchmark-gate --root
    "$PWD"
Passed: True
Cases: 41/41
Expected abstentions: 12
Categories:
  consistency: 5/5 passed, expected abstentions 0
  derivation: 6/6 passed, expected abstentions 5
  workflow: 3/3 passed, expected abstentions 1
  proof_audit: 3/3 passed, expected abstentions 1
  proof_audit_v2: 3/3 passed, expected abstentions 1
```

```
  kalman_recursion: 2/2 passed, expected abstentions 1
  parser_corpus: 3/3 passed, expected abstentions 0
  ast_corpus: 11/11 passed, expected abstentions 0
  typed_ir: 2/2 passed, expected abstentions 2
  industrial_review: 1/1 passed, expected abstentions 1
  release_corpus: 1/1 passed, expected abstentions 0
  release_policy: 1/1 passed, expected abstentions 0
```

Part III

Colleague Workflows

Chapter 9

Workflow 1: Find Mathematical Context

A colleague begins with a vague request: "Where is the likelihood formula?" or "Which note defines the Euler equation we used in that experiment?" This is the moment where an ordinary language model is most tempted to answer from memory. MathDevMCP's first move is different: search the local LaTeX tree and return labels, files, line spans, section paths, and text snippets.

When to use it

Use `search-latex` when the colleague has a topic but not a label. It is especially useful at the beginning of a review, before asking the agent to compare code or audit a derivation. The search result gives the label that later commands use as an anchor.

Command

The example below searches for the Kalman likelihood and returns the matching equation label with file and line provenance.

Listing 9.1: Search workflow output

```
Command: python -m mathdevmcp.cli search-latex Kalman likelihood --root <repo>/
    benchmarks/fixtures --limit 3
[
  {
    "kind": "section",
    "name": "Likelihood derivative notes",
    "file": "doc_longdoc_kalman_retrieval.tex",
    "line_start": 23,
    "line_end": 23,
    "label": null,
    "title": "Likelihood derivative notes",
    "text": "\\section{Likelihood derivative notes}",
```

```
    "block_id": "doc_longdoc_kalman_retrieval.tex:23:section:Likelihood-derivative-notes",
    "section_path": [
      "Likelihood derivative notes"
    ],
    "score": 2
  },
  {
    "kind": "equation",
    "name": "equation",
    "file": "doc_longdoc_kalman_retrieval.tex",
    "line_start": 26,
    "line_end": 31,
    "label": "eq:longdoc-score-contribution",
    "title": null,
    "text": "\\begin{equation}\\label{eq:longdoc-score-contribution}\n\\partial_i \\ell_t\n= -\\frac{1}{2}\\operatorname{tr}(S_t^{-1}\\partial_i S_t)\n+ \\frac{1}{2}v_t' S_t^{-1}(\\partial_i S_t)S_t^{-1}v_t\n- (\\partial_i v_t)'S_t^{-1}v_t.\n\\end{equation}",
    "block_id": "doc_longdoc_kalman_retrieval.tex:26:equation:eq:longdoc-score-contribution",
    "section_path": [
      "Likelihood derivative notes"
    ],
    "score": 2
  },
  {
    "kind": "section",
    "name": "Kalman likelihood Hessian",
    "file": "doc_realistic_kalman_hessian.tex",
    "line_start": 1,
    "line_end": 1,
    "label": null,
    "title": "Kalman likelihood Hessian",
    "text": "\\section{Kalman likelihood Hessian}",
    "block_id": "doc_realistic_kalman_hessian.tex:1:section:Kalman-likelihood-Hessian",
    "section_path": [
      "Kalman likelihood Hessian"
    ],
    "score": 2
  }
]
```


How to read the output

The important fields are the label, file, line span, score, and text. A high score is a useful retrieval signal, not a proof that the result is the only relevant statement. A colleague should inspect the returned section path and then move to the neighborhood extraction workflow if assumptions or notation are likely nearby.

Failure mode

The main failure mode is semantic overreach. A search hit for "likelihood" may find the right formula but not the assumptions that make the formula valid. If the search returns no label, the agent should say so and ask for a narrower query or a document root, rather than fabricating a source.

Agent handoff

An agent handoff should cite the label and provenance from the output rather than paraphrasing from memory. It should not summarize the equation as verified until later workflows check implementation terms or proof obligations.

Chapter 10

Workflow 2: Read a Labeled Neighborhood

Once a label is known, the next question is context. Mathematical documents are rarely self-contained at a single line. A likelihood equation might depend on notation introduced in the preceding align block. A stability condition might depend on a boundary convention. A theorem might be harmless in one model class and false in another.

When to use it

Use `extract-latex-neighborhood` after search and before proof or code claims. It is the right command when the colleague says, "Show me the surrounding assumptions" or "What notation does this equation rely on?"

Command

The example extracts the neighborhood around the state-space likelihood label. It gives the selected block plus nearby paragraphs, line spans, and section metadata.

Listing 10.1: Labeled neighborhood output

```
Command: python -m mathdevmcp.cli extract-latex-neighborhood eq:dept-state-space-
    likelihood --root <repo>/benchmarks/fixtures
{
  "file": "doc_department_state_space.tex",
  "line_start": 21,
  "line_end": 23,
  "context_start": 11,
  "context_end": 23,
  "paragraphs": [
    {
      "line_start": 11,
      "line_end": 19,
```

```
      "text": "\\begin{align}\\label{eq:dept-state-space-recursion}\nx^{\\mathrm{pred}}
          _t &= F_t x_{t-1} \\\\\n\\PredCov &= F_t P_{t-1}F_t' + Q_t \\\\\n\\Innov &=
          y_t - H_t x^{\\mathrm{pred}}_t \\\\\n\\InnovCov &= H_t \\PredCov H_t' + R_t
          \\\\\nK_t &= \\PredCov H_t' \\InnovCov^{-1} \\\\\nx_t &= x^{\\mathrm{pred}}_t
          + K_t \\Innov \\\\\nP_t &= \\PredCov - K_t H_t \\PredCov\n\\end{align}"
    },
    {
      "line_start": 21,
      "line_end": 23,
      "text": "\\begin{equation}\\label{eq:dept-state-space-likelihood}\n\\ell_t = -\\
          frac{1}{2}\\left(\\log\\det \\InnovCov + \\Innov' \\InnovCov^{-1}\\Innov\\
          right).\n\\end{equation}"
    }
  ],
  "label": "eq:dept-state-space-likelihood",
  "kind": "equation",
  "block_id": "doc_department_state_space.tex:21:equation:eq:dept-state-space-likelihood
      ",
  "section_path": [
    "Sanitized state-space audit slice"
  ]
}
```


How to read the output

Read the neighborhood as a source packet. The label is the anchor, the file and line numbers are where a colleague should inspect the original document, and the paragraphs are the local assumptions or notation the agent may cite. If the neighborhood omits an assumption that the proof needs, that is a signal to route the claim to human review.

Failure mode

Neighborhoods are local. They do not prove that a global assumption is present elsewhere in the document. The agent should use them to ground a claim, then say what is still missing.

Agent handoff

A good handoff says: "I found this label and its immediate context; the next check is whether the implementation carries the required operations." That handoff sets up the comparison workflow without pretending the context alone is certification.

Chapter 11

Workflow 3: Compare Document and Code

Code–document drift is one of the highest-value release use cases. A colleague may trust a derivation in a note while the code has silently changed. The `compare-label-code` workflow asks a smaller, sharper question: do the required operations from a labeled document block appear in a chosen code file?

When to use it

Use this workflow when the review question has a concrete code target. It is not for broad semantic equivalence. It is for surfacing missing terms, suspicious renamings, or stale implementations that deserve human attention.

Command

The example compares the state-space likelihood against a deliberately incomplete implementation. The document asks for log determinant and inverse or solve structure; the code has the log determinant but misses the solve term.

Listing 11.1: Code–document comparison output

```
Command: python -m mathdevmcp.cli compare-label-code eq:dept-state-space-likelihood <repo>/benchmarks/fixtures/doc_department_state_space_missing_solve.py --root <repo>/benchmarks/fixtures --required-terms logdet,solve --paragraph-context
{
  "status": "mismatch",
  "reason": "Some required document terms were not found in the code text.",
  "doc_terms": [
    "logdet",
    "solve"
  ],
  "code_terms": [
    "def",
```

```
    "kalman_filter_scan",
    "jnp",
    "lax",
    "params",
    "step",
    "carry",
    "obs",
    "x_pred",
    "p_pred",
    "innovation",
    "sign",
    "logdet",
    "linalg",
    "slogdet",
    "x_filt",
    "p_filt",
    "ll_t",
    "return",
    "scan",
    "initial_state"
  ],
  "missing_in_code": [
    "solve"
  ],
  "findings": [
    {
      "kind": "matched_term",
      "term": "logdet",
      "present_in_code": true,
      "severity": "required"
    },
    {
      "kind": "missing_term",
      "term": "solve",
      "present_in_code": false,
      "severity": "required"
    },
    {
      "kind": "extra_code_terms",
      "terms": [
        "carry",
        "def",
        "initial_state",
        "innovation",
        "jnp",
        "kalman_filter_scan",
        "lax",
        "linalg",
        "ll_t",
        "obs",
        "p_filt",
        "p_pred",
        "params",
        "return",
        "scan",
        "sign",
        "slogdet",
        "step",
        "x_filt",
```

```
        "x_pred"
      ],
      "severity": "audit_only"
    }
  ],
  "extra_in_code": [
    "carry",
    "def",
    "initial_state",
    "innovation",
    "jnp",
    "kalman_filter_scan",
... truncated for report ...
```

How to read the output

The status is `mismatch`; that does not mean MathDevMCP proved the code wrong. It means the configured required term was not found in the inspected code text. The useful fields are `matched_term`, `missing_term`, `code_path`, and the document provenance. Those fields are enough to write a precise review ticket.

Failure mode

A term check can miss semantic equivalents or be fooled by a misleading variable name. This is why the report calls it a drift detector, not a proof. The next step is to inspect the implementation and, if needed, add a benchmark that captures the intended operation more robustly.

Agent handoff

The agent should say: "The document label requires `solve`; the selected implementation did not expose that term. Please review whether the quadratic form is implemented with a numerically appropriate solve or an equivalent operation." That is useful, bounded, and honest.

Chapter 12

Workflow 4: Audit a Derivation

The v2 audit workflow is the product's strongest guard against false confidence. It combines parser policy, typed obligations, shape diagnostics, numeric suggestions, backend evidence, and verification-boundary text. It is the right surface when the colleague asks, "Is this certified, or just plausible?"

When to use it

Use `audit-derivation-v2-label` when the label contains a mathematical claim whose assumptions matter. Matrix inverses, log determinants, gradients, expectations, and stability conditions all carry hidden obligations. The audit exposes those obligations instead of burying them in prose.

Command

The example audits the state-space likelihood. It finds high-priority actions: state or verify invertibility, square-matrix, conformability, log-det domain, and linear-solve residual constraints.

Listing 12.1: Derivation audit output

```
Command: python -m mathdevmcp.cli audit-derivation-v2-label eq:dept-state-space-
    likelihood --root <repo>/benchmarks/fixtures --summary-only
{
  "label": "eq:dept-state-space-likelihood",
  "doc_root": "<repo>/benchmarks/fixtures",
  "status": "unverified",
  "reason": "At least one obligation remains unverified or diagnostic-only.",
  "counts": {
    "total": 1,
    "verified": 0,
    "mismatch": 0,
    "unverified": 1,
    "inconclusive": 0
```

```
  },
  "high_priority_actions": [
    {
      "kind": "state_or_verify_missing_constraint",
      "target": "invertibility_required",
      "severity": "high",
      "reason": "Matrix inverse or solve notation requires an invertible or positive-
          definite operand.",
      "obligation_id": "obligation_1"
    },
    {
      "kind": "state_or_verify_missing_constraint",
      "target": "square_matrix_required",
      "severity": "high",
      "reason": "Determinant and log determinant require a square matrix operand.",
      "obligation_id": "obligation_1"
    },
    {
      "kind": "state_or_verify_missing_constraint",
      "target": "conformable_product_required",
      "severity": "high",
      "reason": "Transpose and matrix product notation require conformable dimensions.",
      "obligation_id": "obligation_1"
    },
    {
      "kind": "human_formalization_or_review",
      "severity": "high",
      "reason": "Typed obligation has missing assumptions or dimension constraints.",
      "obligation_id": "obligation_1"
    },
    {
      "kind": "logdet_domain_check",
      "target": "determinant/logdet operand",
      "severity": "high",
      "reason": "Log determinant requires square positive-definite or otherwise valid
          matrix input.",
      "obligation_id": "obligation_1"
    },
    {
      "kind": "linear_solve_residual_check",
      "target": "inverse/solve operand",
      "severity": "high",
      "reason": "Inverse notation should be checked with solve residuals and
          conditioning diagnostics.",
      "obligation_id": "obligation_1"
    }
  ],
  "obligations": [
    {
      "id": "obligation_1",
      "label": "eq:dept-state-space-likelihood",
      "status": "unverified",
      "reason": "The obligation has missing shape, dimension, or regularity constraints
          .",
      "evidence_kind": "diagnostic_bundle",
      "diagnostic_only": true,
      "verification_boundary": "The obligation has missing shape, dimension, or
          regularity constraints. A deterministic backend certificate is required before
           this can become verified.",
```

```
      "route": "human_review",
      "missing_constraints": [
        {
          "kind": "invertibility_required",
          "target": "inverse operand",
          "reason": "Matrix inverse or solve notation requires an invertible or positive
              -definite operand.",
          "status": "missing_assumption",
          "source": "unresolved_constructs"
        },
        {
          "kind": "square_matrix_required",
          "target": "determinant/logdet operand",
          "reason": "Determinant and log determinant require a square matrix operand.",
          "status": "missing_assumption",
          "source": "unresolved_constructs"
        },
... truncated for report ...
```


How to read the output

The key field is the verification boundary. If the status is `unverified` or `diagnostic_only`, the tool is telling the agent not to claim a proof. The high-priority actions are the review checklist. A colleague can turn them into assumptions to state, tests to add, or formal obligations to discharge.

Failure mode

The audit can be verbose. The danger is not verbosity; the danger is summarizing away the boundary. A release-quality agent must keep the words "unverified", "diagnostic", or "requires human review" visible when those statuses appear.

Agent handoff

The handoff should separate evidence from certification: "The parser located the label and the audit identified missing matrix constraints. No deterministic backend certificate is present, so this remains a human-review item."

Chapter 13

Workflow 5: Build an Implementation Brief

The implementation brief is the handoff format for a busy colleague. It bundles search, label selection, context extraction, consistency checking, and optional derivation evidence into one object. It is the report a teammate can paste into a review thread before opening the code.

When to use it

Use `implementation-brief` when another person or agent needs a grounded starting point. It is especially useful before a pull request review, a maintenance ticket, or a focused refactor of mathematical code.

Command

The example asks for a state-space likelihood brief against the same incomplete implementation used above. The brief selects the relevant label, includes context, and reports the consistency mismatch.

Listing 13.1: Implementation brief output

```
Command: python -m mathdevmcp.cli implementation-brief state space likelihood <repo>/
    benchmarks/fixtures/doc_department_state_space_missing_solve.py --root <repo>/
    benchmarks/fixtures --required-terms logdet,solve --limit 2
{
  "query": "state space likelihood",
  "doc_root": "<repo>/benchmarks/fixtures",
  "code_path": "<repo>/benchmarks/fixtures/doc_department_state_space_missing_solve.py",
  "search_results": [
    {
      "kind": "equation",
      "name": "equation",
      "file": "doc_department_state_space.tex",
```

```
      "line_start": 21,
      "line_end": 23,
      "label": "eq:dept-state-space-likelihood",
      "title": null,
      "text": "\\begin{equation}\\label{eq:dept-state-space-likelihood}\n\\ell_t = -\\
          frac{1}{2}\\left(\\log\\det \\InnovCov + \\Innov' \\InnovCov^{-1}\\Innov\\
          right).\n\\end{equation}",
      "block_id": "doc_department_state_space.tex:21:equation:eq:dept-state-space-
          likelihood",
      "section_path": [
        "Sanitized state-space audit slice"
      ],
      "score": 3
    },
    {
      "kind": "section",
      "name": "Sanitized state-space audit slice",
      "file": "doc_department_state_space.tex",
      "line_start": 1,
      "line_end": 1,
      "label": null,
      "title": "Sanitized state-space audit slice",
      "text": "\\section{Sanitized state-space audit slice}",
      "block_id": "doc_department_state_space.tex:1:section:Sanitized-state-space-audit-
          slice",
      "section_path": [
        "Sanitized state-space audit slice"
      ],
      "score": 2
    }
  ],
  "selected_label": "eq:dept-state-space-likelihood",
  "status": "mismatch",
  "cache": {
    "path": null,
    "hit": false
  },
  "checks": {
    "consistency": {
      "status": "mismatch",
      "reason": "Some required document terms were not found in the code text.",
      "doc_terms": [
        "logdet",
        "solve"
      ],
      "code_terms": [
        "def",
        "kalman_filter_scan",
        "jnp",
        "lax",
        "params",
        "step",
        "carry",
        "obs",
        "x_pred",
        "p_pred",
        "innovation",
        "sign",
        "logdet",
```

```
        "linalg",
        "slogdet",
        "x_filt",
        "p_filt",
        "ll_t",
        "return",
        "scan",
        "initial_state"
      ],
      "missing_in_code": [
        "solve"
      ],
      "findings": [
        {
          "kind": "matched_term",
          "term": "logdet",
... truncated for report ...
```

How to read the output

The brief's `selected_label` tells the reviewer what document claim was used. The `checks.consistency.status` field tells whether the required terms appeared. The context excerpt gives the exact mathematical block. This is enough for a reviewer to reproduce the check without trusting the agent's memory.

Failure mode

The brief is only as good as the query and required terms. If the query selects the wrong label, or if the required terms are too weak, the brief can be uninformative. The right response is to rerun with a sharper query or a better term set, not to inflate the conclusion.

Agent handoff

A good handoff reads like a compact review note: selected label, code file, status, missing terms, and next action. It should avoid broad claims such as "the implementation is wrong" unless a human review or stronger backend evidence supports that claim.

Part IV

Case Studies

Chapter 14

Kalman State-Space Likelihood

Colleague scenario

A colleague is reviewing a Kalman filter implementation before using it in a forecasting experiment. The note defines a Gaussian likelihood with $\log\det S_t$ and S_t^{-1} terms. The implementation has been optimized with a scan loop, and the reviewer wants to know whether the code still exposes the solve-like operation implied by the derivation.

Fixture and command

The public fixture uses the state-space document and a deliberately incomplete missing-solve implementation. The command compares the likelihood label against required terms `logdet,solve`.

MCP conversation

```
User:
  Check whether the state-space likelihood label matches the Kalman code.
  Look specifically for logdet and solve-like structure.
Agent:
  I will anchor the review on the document label before judging the code.
Tool: search_latex(root, "state space likelihood logdet solve")
Tool: compare_label_code(root, "eq:dept-state-space-likelihood",
                         code, required_terms=["logdet", "solve"],
                         paragraph_context=true)
Agent:
  The result is mismatch: logdet is visible, solve is missing. This is a
  drift signal, not a proof of invalidity. Review whether an equivalent
  factorization implements the quadratic solve.
```

Listing 14.1: Kalman likelihood comparison evidence

```
Command: python -m mathdevmcp.cli compare-label-code eq:dept-state-space-likelihood <repo>/benchmarks/fixtures/doc_department_state_space_missing_solve.py --root <repo>/benchmarks/fixtures --required-terms logdet,solve --paragraph-context
Label: eq:dept-state-space-likelihood
Code file: doc_department_state_space_missing_solve.py
Status: mismatch
Reason: Some required document terms were not found in the code text.
Required terms: logdet,solve
Matched terms: ['logdet']
Missing terms: ['solve']
Provenance: doc_department_state_space.tex:21-23
Document excerpt: \begin{equation}\label{eq:dept-state-space-likelihood} \ell_t = -\frac{1}{2}\left(\log\det \InnovCov + \Innov' \InnovCov^{-1}\Innov\right). \end{equation}
```

Output to inspect

The output is intentionally uncomfortable: status `mismatch`, matched term `logdet`, and missing term `solve`. That is exactly the kind of result that makes the product useful. It does not bury a missing numerical operation under a friendly summary. It gives a reviewer the label, file, line span, code path, and missing term.

Interpretation

The mathematical issue is numerical, not merely textual. A likelihood written with S_t^{-1} should usually be implemented through a solve or factorization, not by casually dropping the quadratic solve. The tool cannot prove the code is wrong from the missing token alone, but it can make the review precise: inspect the innovation quadratic form and decide whether a stable solve is present under another name.

Next action

The colleague should open the implementation, check how the quadratic form is computed, and add or fix a test that exercises conditioning of S_t. If the code intentionally uses an equivalent factorization, the benchmark should be updated with an operation extractor or required term that recognizes the chosen idiom.

Boundary

MathDevMCP reports a drift signal, not a theorem. It does not certify the Kalman likelihood, and it does not prove the implementation is invalid. It prevents an agent from saying "the code matches the document" when a required solve-like operation is not visible.

Chapter 15

HMC Leapfrog and Hamiltonian Flow

Colleague scenario

A Bayesian computation colleague is checking an HMC kernel. The document gives a leapfrog update and a Hamiltonian. The code has gradients, half-steps, kinetic energy, and a returned Hamiltonian value. The review question is not "does this sampler converge?" It is narrower: does the implementation visibly carry the documented leapfrog and Hamiltonian structure?

Fixture and command

The fixture uses `doc_department_bayesian_hmc.tex` and `doc_department_hmc_jax.py`. The comparison asks for `grad` and `hamiltonian` under label `eq:dept-hmc-leapfrog`.

MCP conversation

```
User:
  Does the HMC implementation visibly carry the leapfrog and Hamiltonian
  structure from the note?
Agent:
  I will compare the leapfrog label against the HMC code for gradient and
  Hamiltonian terms.
Tool: compare_label_code(root, "eq:dept-hmc-leapfrog", code,
                         required_terms=["grad", "hamiltonian"])
Agent:
  The structural check is consistent. I can say the selected code is not
  obviously stale with respect to those terms, but I cannot certify detailed
  balance, acceptance behavior, or convergence from this check.
```

Listing 15.1: HMC leapfrog comparison evidence

```
Command: python -m mathdevmcp.cli compare-label-code eq:dept-hmc-leapfrog <repo>/
    benchmarks/fixtures/doc_department_hmc_jax.py --root <repo>/benchmarks/fixtures --
    required-terms grad,hamiltonian --paragraph-context
Label: eq:dept-hmc-leapfrog
Code file: doc_department_hmc_jax.py
Status: consistent
Reason: All required document terms were found in the code text.
Required terms: grad,hamiltonian
Matched terms: ['grad', 'hamiltonian']
Missing terms: []
Provenance: doc_department_bayesian_hmc.tex:13-17
Document excerpt: \begin{align}\label{eq:dept-hmc-leapfrog} p_{n+1/2} &= p_n + \frac{\
    epsilon}{2}\nabla_\theta \logpost \\ \theta_{n+1} &= \theta_n + \epsilon M^{-1}p_{n
    +1/2} \\ p_{n+1} &= p_{n+1/2} + \frac{\epsilon}{2}\nabla_\theta \logpost \end{align}
```

Output to inspect

The status is `consistent`: both required terms appear. The output also keeps the provenance of the align block so a colleague can see the three leapfrog lines rather than trusting a paraphrase.

Interpretation

This is a positive structural check. It says the obvious ingredients for the documented leapfrog are present in the inspected code. It is useful before a deeper review because it quickly rules out a stale file that lacks gradients or Hamiltonian energy entirely.

Next action

The next review step is to inspect signs, step ordering, mass matrix semantics, and Metropolis correction. If the claim is about stationarity or convergence, route it to human review or a dedicated proof/audit workflow; do not treat a term match as a Markov-chain proof.

For a release note or pull-request handoff, the useful sentence is: the code is not obviously stale with respect to the documented leapfrog structure, but the probabilistic correctness argument remains outside this check.

Boundary

MathDevMCP does not certify HMC correctness, detailed balance, ergodicity, or acceptance-rate behavior here. It confirms that specific document-grounded operations are visible and keeps the remaining probabilistic claims outside the certificate boundary.

Maintainers adding richer HMC coverage should add a seeded sign-error or missing Metropolis-correction case before changing the release claim.

Chapter 16

Macro Filter Multi-File Corpus

Colleague scenario

The macro-filter case resembles a real internal note: a main LaTeX file defines macros, includes a model file, and repeats notation across state equations. A colleague has an implementation that computes prediction and likelihood terms but may have omitted the Kalman gain update. The question is whether the tool can follow the included document and expose the omission.

Fixture and command

The fixture uses a macro-filter main document, an included model file, and a deliberately incomplete missing-gain implementation. The comparison below checks the likelihood label for `logdet`, `solve`, and `K_t`.

MCP conversation

```
User:
  This macro filter is split across included TeX files. Check whether the
  implementation has the likelihood terms and the Kalman gain update.
Agent:
  I will let the parser resolve the included model file, then compare the
  selected label to the implementation.
Tool: search_latex(root, "macro filter likelihood Kalman gain")
Tool: compare_label_code(root, "eq:macro-filter-likelihood", code,
                         required_terms=["logdet", "solve", "K_t"])
Agent:
  The comparison finds logdet and solve but not K_t. The likely review
  question is whether this function is intentionally prediction-only or has
  dropped the gain/update step.
```

Listing 16.1: Macro filter comparison evidence

```
Command: python -m mathdevmcp.cli compare-label-code eq:macro-filter-likelihood <repo>/
    benchmarks/fixtures/doc_macro_filter_missing_gain.py --root <repo>/benchmarks/
    fixtures --required-terms logdet,solve,K_t --paragraph-context
Label: eq:macro-filter-likelihood
Code file: doc_macro_filter_missing_gain.py
Status: mismatch
Reason: Some required document terms were not found in the code text.
Required terms: logdet,solve,K_t
Matched terms: ['logdet', 'solve']
Missing terms: ['K_t']
Provenance: doc_macro_filter_model.tex:20-23
Document excerpt: \begin{equation} \label{eq:macro-filter-likelihood} \ell_t = -\frac
    {1}{2}\log\det\cov - \frac{1}{2}\innov^\top \cov^{-1}\innov \end{equation}
```

Output to inspect

The output reports `mismatch`: `logdet` and `solve` are present, while `K_t` is missing. The provenance points into the included model file, not only the main file, which matters because many real documents are modular.

Interpretation

This case demonstrates two product behaviors at once. First, the parser handles the multi-file LaTeX structure well enough to locate the model equation. Second, the code comparison can distinguish a likelihood calculation that contains linear algebra terms from a complete filter update that carries the gain.

That distinction is exactly where colleagues lose time in practice: a partial routine can look mathematically serious because it contains the log determinant and solve, while still omitting the update needed by the model description.

Next action

The colleague should inspect whether the implementation intentionally returns a prediction-only likelihood or whether it should update state and covariance. If the implementation is intentionally partial, the document or function name should say so. If not, add the missing gain/update logic and a regression test.

If this pattern recurs in a project, make `K_t` or the chosen gain-update idiom part of the domain benchmark rather than relying on a one-off review comment.

Boundary

The tool does not infer all macro semantics or prove the filter recursion. It surfaces a targeted mismatch in a realistic multi-file setting and leaves the modeling decision to the reviewer.

Chapter 17

DSGE Euler Equation

Colleague scenario

A macro-finance colleague has a compact Euler equation in the notes and a small residual function in code. The review question is whether the implementation still computes an expectation of a stochastic discount factor times a return, not whether the model is calibrated, solved, or empirically valid.

Fixture and command

The fixture uses `doc_sanitized_dsge_macro_finance.tex` and `doc_sanitized_dsge_macro_finance.py`. The comparison checks `eq:dept-euler-equation` for `expectation` and `euler_residual`.

MCP conversation

```
User:
  In the DSGE example, check that the code still represents the Euler
  residual from the note. Do not make calibration claims.
Agent:
  I will compare the Euler-equation label to the residual code for the
  configured structural terms.
Tool: compare_label_code(root, "eq:dept-euler-equation", code,
                         required_terms=["expectation", "euler_residual"])
Agent:
  The comparison is consistent for those terms. The code carries the named
  residual structure, but calibration, timing, and equilibrium validity remain
  outside this evidence.
```

Listing 17.1: DSGE Euler comparison evidence

```
Command: python -m mathdevmcp.cli compare-label-code eq:dept-euler-equation <repo>/
    benchmarks/fixtures/doc_sanitized_dsge_macro_finance.py --root <repo>/benchmarks/
    fixtures --required-terms expectation,euler_residual --paragraph-context
```

```
Label: eq:dept-euler-equation
Code file: doc_sanitized_dsge_macro_finance.py
Status: consistent
Reason: All required document terms were found in the code text.
Required terms: expectation,euler_residual
Matched terms: ['expectation', 'euler_residual']
Missing terms: []
Provenance: doc_sanitized_dsge_macro_finance.tex:3-5
Document excerpt: \begin{equation}\label{eq:dept-euler-equation} E_t[m_{t+1} R_{t+1}] -
    1 = 0. \end{equation}
```

Output to inspect

The status is `consistent`, and the matched terms show that the implementation contains both the expectation operator and the residual concept. The provenance points to the Euler equation in the sanitized macro-finance fixture.

Interpretation

This is the kind of small but valuable check that prevents stale glue code. It does not decide the economic model; it ensures that the code being reviewed at least carries the structural ingredients named in the document.

The value is speed with restraint. The reviewer can quickly move past the "wrong file" and "missing residual" class of errors while keeping the economic interpretation under expert review.

Next action

The colleague should next review calibration assumptions, timing conventions, state definitions, and units. If those are part of the release claim, add fixture labels and expected abstentions that force the tool to report missing assumptions.

For production use, this is where a team should add project-specific labels for household timing, asset returns, and shock normalization if those conventions are common sources of errors.

Boundary

MathDevMCP does not solve the DSGE model, estimate parameters, or validate an equilibrium. It gives document-grounded evidence that the inspected residual function has the expected structural terms.

Chapter 18

Stochastic Volatility Likelihood

Colleague scenario

A colleague is reviewing a stochastic-volatility likelihood implementation. The document has a transition equation for latent volatility and a conditional likelihood for observations. The code computes an innovation-like likelihood term, and the reviewer wants to check that the expected likelihood structure is visible.

Fixture and command

The fixture uses `doc_sanitized_stochastic_volatility.tex` and `doc_sanitized_stochastic_volatility.py`. The comparison checks `eq:dept-sv-likelihood` for `innovation` and `posterior_or_likelihood`.

MCP conversation

```
User:
  Check whether this stochastic-volatility likelihood code has the likelihood
  path described in the document.
Agent:
  I will compare the likelihood label against the code for innovation and
  likelihood/posterior structure.
Tool: compare_label_code(root, "eq:dept-sv-likelihood", code,
                         required_terms=["innovation",
                                         "posterior_or_likelihood"])
Agent:
  The selected structure is visible. I would still route latent-prior,
  Jacobian, and inference-method assumptions to human review.
```

Listing 18.1: Stochastic volatility comparison evidence

```
Command: python -m mathdevmcp.cli compare-label-code eq:dept-sv-likelihood <repo>/
    benchmarks/fixtures/doc_sanitized_stochastic_volatility.py --root <repo>/benchmarks/
    fixtures --required-terms innovation,posterior_or_likelihood --paragraph-context
```

```
Label: eq:dept-sv-likelihood
Code file: doc_sanitized_stochastic_volatility.py
Status: consistent
Reason: All required document terms were found in the code text.
Required terms: innovation,posterior_or_likelihood
Matched terms: ['innovation', 'posterior_or_likelihood']
Missing terms: []
Provenance: doc_sanitized_stochastic_volatility.tex:7-9
Document excerpt: \begin{equation}\label{eq:dept-sv-likelihood} \log p(y_t\mid h_t) = -\
    frac{1}{2}\left(h_t + y_t^2\exp(-h_t)\right) + C. \end{equation}
```

Output to inspect

The status is `consistent`; both required terms are found. The output gives the document label, code file, matched terms, and the likelihood equation excerpt.

Interpretation

This result is useful because it confirms the code has a likelihood-bearing path that corresponds to the document. It also tells the reviewer what remains out of scope: latent prior assumptions, filtering/inference strategy, Jacobians, and parameter constraints are not certified by this term check.

The example is intentionally narrow. It is strong enough to catch a stale file with no likelihood path, but too weak to approve a full inference algorithm.

Next action

The colleague should inspect whether the transition variance, parameter bounds, and inference algorithm match the intended model. If missing-Jacobian or prior review is a recurring issue, add a seeded false-confidence case to the benchmark corpus.

Teams using particle filters, variational approximations, or MCMC for this model should add separate fixtures for those inference choices, because the likelihood term alone will not capture them.

Boundary

MathDevMCP does not certify the stochastic-volatility likelihood or posterior inference. It confirms selected structure and keeps latent-model assumptions in the human-review lane unless more evidence is supplied.

Chapter 19

SDE and PDE Numerics

Colleague scenario

A numerical methods colleague is checking whether a simulation step reflects the Euler–Maruyama update in the note and whether the code carries an explicit stability condition. The aim is to catch silent drift between mathematical discretization and implementation before experiments are trusted.

Fixture and command

The fixture uses `doc_sanitized_sde_pde_numerics.tex` and `doc_sanitized_sde_pde_numerics.py`. The comparison checks `eq:dept-euler-maruyama` for `drift`, `diffusion`, and `stability_condition`.

MCP conversation

```
User:
  Before we trust these simulations, check whether the code carries the
  Euler-Maruyama drift, diffusion, and stability hook from the note.
Agent:
  I will anchor on the Euler-Maruyama label and inspect the selected code.
Tool: compare_label_code(root, "eq:dept-euler-maruyama", code,
                         required_terms=["drift", "diffusion",
                                         "stability_condition"])
Agent:
  The structural terms are present. This supports an implementation-screening
  pass, not a convergence or stability proof.
```

Listing 19.1: SDE/PDE numerics comparison evidence

```
Command: python -m mathdevmcp.cli compare-label-code eq:dept-euler-maruyama <repo>/
    benchmarks/fixtures/doc_sanitized_sde_pde_numerics.py --root <repo>/benchmarks/
    fixtures --required-terms drift,diffusion,stability_condition --paragraph-context
Label: eq:dept-euler-maruyama
```

```
Code file: doc_sanitized_sde_pde_numerics.py
Status: consistent
Reason: All required document terms were found in the code text.
Required terms: drift,diffusion,stability_condition
Matched terms: ['drift', 'diffusion', 'stability_condition']
Missing terms: []
Provenance: doc_sanitized_sde_pde_numerics.tex:3-5
Document excerpt: \begin{equation}\label{eq:dept-euler-maruyama} X_{n+1} = X_n + b(X_n)\
    Delta t + \sigma(X_n)\sqrt{\Delta t}\xi_n. \end{equation}
```

Output to inspect

The status is `consistent`: the implementation exposes drift, diffusion, and a stability-condition term. The provenance points to the Euler–Maruyama formula.

Interpretation

This is a structural release check, not a numerical analysis proof. It tells the reviewer that the code carries the named pieces of the discretization. It does not tell the reviewer that the step size is appropriate for every SDE, that the PDE residual is stable, or that convergence assumptions hold.

That boundary makes the check useful in a lab setting: it catches missing drift, diffusion, or stability hooks before anyone spends time interpreting simulated paths as evidence about the continuous model.

Next action

The colleague should inspect the time-step bound, units, boundary conditions, and randomness convention. If a stability argument is important to the release, add a proof-audit or typed-obligation example that forces missing assumptions to surface.

For code review, the best follow-up is often a small stress test near the documented stability limit, because such tests expose convention mismatches that token checks cannot see.

Boundary

MathDevMCP does not prove convergence or stability. It keeps the implementation review anchored to the documented discretization and makes missing terms visible early.

Chapter 20

ML and LLM Objective Functions

Colleague scenario

An ML colleague is reviewing objective code after several training-loop edits. The document states a negative log-likelihood loss and its gradient. The code uses an automatic differentiation function named `grad`, but the report is asked to check for the more explicit term `gradient`. This is a useful stress test for term-level review.

Fixture and command

The fixture uses `doc_sanitized_ml_llm_objective.tex` and `doc_sanitized_ml_llm_objective.py`. The comparison checks `eq:dept-ml-gradient` for `gradient` and `loss`.

MCP conversation

```
User:
  The ML code uses autodiff. Check whether the documented gradient objective
  is visible, but do not silently equate grad with the full derivation.
Agent:
  I will run the configured term check and report literal matches.
Tool: compare_label_code(root, "eq:dept-ml-gradient", code,
                         required_terms=["gradient", "loss"])
Agent:
  The result is mismatch: loss is visible, gradient is not matched exactly.
  The code may use grad or another idiom, so the next step is to decide
  whether the benchmark should recognize that operation explicitly.
```

Listing 20.1: ML objective comparison evidence

```
Command: python -m mathdevmcp.cli compare-label-code eq:dept-ml-gradient <repo>/
    benchmarks/fixtures/doc_sanitized_ml_llm_objective.py --root <repo>/benchmarks/
    fixtures --required-terms gradient,loss --paragraph-context
Label: eq:dept-ml-gradient
Code file: doc_sanitized_ml_llm_objective.py
```

```
Status: mismatch
Reason: Some required document terms were not found in the code text.
Required terms: gradient,loss
Matched terms: ['loss']
Missing terms: ['gradient']
Provenance: doc_sanitized_ml_llm_objective.tex:7-9
Document excerpt: \begin{equation}\label{eq:dept-ml-gradient} \nabla_\theta L(\theta) =
    -\sum_i \nabla_\theta \log p_\theta(y_i\mid x_i). \end{equation}
```

Output to inspect

The status is `mismatch`: `loss` is present, but `gradient` is missing. The code does contain `grad` and `gradient_value`, yet the configured required term is not present exactly. That is not a failure of honesty; it is a reminder that term checks are intentionally literal.

Interpretation

This example is good for selling the product because it shows both value and limits. The tool catches that the configured review vocabulary does not line up with the code. A reviewer can decide whether to accept `grad` as the implementation idiom, change the required term list, or add operation extraction that recognizes automatic differentiation.

Next action

The colleague should review sign conventions, reduction over the batch, and the automatic-differentiation call. If the code is correct, update the benchmark to look for the intended operation rather than a single English token.

Boundary

MathDevMCP does not prove the training objective is correct, and it does not know that `grad` is semantically equivalent to the full documented gradient in all contexts. It reports the literal evidence and asks the reviewer to sharpen the contract.

Chapter 21

Bayesian ELBO and Variational Inference

Colleague scenario

A Bayesian ML colleague is reviewing a variational-inference implementation. The document names the ELBO and a reparameterization gradient. The code computes an `elbo_value`, but the configured term check asks for `elbo` exactly. This is another realistic review problem: naming conventions are close but not identical.

Fixture and command

The fixture uses `doc_sanitized_bayesian_elbo_vi.tex` and `doc_sanitized_bayesian_elbo_vi.py`. The comparison checks `eq:dept-elbo` for `expectation` and `elbo`.

MCP conversation

```
User:
  Check the ELBO implementation. If the code only has a nearby name such as
  elbo_value, keep that distinction visible.
Agent:
  I will run the literal configured review terms against the ELBO label.
Tool: compare_label_code(root, "eq:dept-elbo", code,
                         required_terms=["expectation", "elbo"])
Agent:
  The result is mismatch: expectation is present, elbo is not matched exactly.
  I see clues that may be relevant, but I will not turn those clues into a
  certification without a sharper operation extractor or human review.
```

Listing 21.1: ELBO comparison evidence

```
Command: python -m mathdevmcp.cli compare-label-code eq:dept-elbo <repo>/benchmarks/
    fixtures/doc_sanitized_bayesian_elbo_vi.py --root <repo>/benchmarks/fixtures --
    required-terms expectation,elbo --paragraph-context
Label: eq:dept-elbo
Code file: doc_sanitized_bayesian_elbo_vi.py
Status: mismatch
Reason: Some required document terms were not found in the code text.
Required terms: expectation,elbo
Matched terms: ['expectation']
Missing terms: ['elbo']
Provenance: doc_sanitized_bayesian_elbo_vi.tex:3-5
Document excerpt: \begin{equation}\label{eq:dept-elbo} \mathcal L(\lambda)=E_{q_\lambda(
    z)}[\log p(y,z)-\log q_\lambda(z)]. \end{equation}
```

Output to inspect

The status is `mismatch`: `expectation` is present while `elbo` is missing. The extra terms include `elbo_value` and `reparameterization_gradient`, which gives the reviewer a useful clue without silently converting that clue into a match.

Interpretation

This is a disciplined false-confidence example. A loose agent might say "ELBO is there" because it sees `elbo_value`. MathDevMCP reports the exact configured term result and leaves the interpretation visible. That lets the team decide whether token matching is enough or whether AST/operation extraction should be extended for this domain.

The point is not to embarrass the checker for being literal. The point is to make the review contract visible enough that the team can improve it deliberately instead of accepting a fuzzy match.

Next action

The colleague should inspect entropy terms, support assumptions, and whether the reparameterization path matches the document. If the naming convention is accepted, tune the release evidence so the benchmark captures the intended semantic operation.

For future work, the natural extension is an operation extractor that recognizes ELBO assembly patterns rather than only the spelling of one variable.

Boundary

MathDevMCP does not certify the ELBO derivation, variational-family support, or gradient estimator validity. It gives a reproducible review signal and keeps the proof

boundary explicit.

Chapter 22

Computational Physics MCMC

Colleague scenario

A computational physics colleague is checking MCMC code that computes a Hamiltonian acceptance ratio. The document states an acceptance formula, and the code computes Hamiltonian energies and a log acceptance probability. The review question is whether those core ingredients are visible.

Fixture and command

The fixture uses the sanitized computational-physics MCMC document and its paired code file. The comparison checks the acceptance-ratio label for `hamiltonian` and `gradient`.

MCP conversation

```
User:
  Does the MCMC implementation carry the Hamiltonian and gradient ingredients
  from the acceptance-ratio note?
Agent:
  I will compare the acceptance-ratio label to the code for those structural
  terms only.
Tool: compare_label_code(root, "eq:dept-acceptance-ratio", code,
                         required_terms=["hamiltonian", "gradient"])
Agent:
  The selected terms are present. This is useful screening evidence, but it
  does not certify reversibility, detailed balance, or ergodicity.
```

Listing 22.1: Computational physics MCMC comparison evidence

```
Command: python -m mathdevmcp.cli compare-label-code eq:dept-acceptance-ratio <repo>/
    benchmarks/fixtures/doc_sanitized_computational_physics_mcmc.py --root <repo>/
    benchmarks/fixtures --required-terms hamiltonian,gradient --paragraph-context
Label: eq:dept-acceptance-ratio
Code file: doc_sanitized_computational_physics_mcmc.py
```

```
Status: consistent
Reason: All required document terms were found in the code text.
Required terms: hamiltonian,gradient
Matched terms: ['hamiltonian', 'gradient']
Missing terms: []
Provenance: doc_sanitized_computational_physics_mcmc.tex:3-5
Document excerpt: \begin{equation}\label{eq:dept-acceptance-ratio} \alpha(x,x') = \min
    \{1,\exp(-H(x')+H(x))\}. \end{equation}
```

Output to inspect

The status is `consistent`: Hamiltonian and gradient terms are found. The output also shows the acceptance-ratio label and document excerpt.

Interpretation

This is a positive release-facing check. It confirms the implementation has the Hamiltonian energy and gradient structure expected by the note. It does not settle detailed balance, reversibility, or convergence. Those require stronger mathematical arguments and domain review.

This is the right level of automation for an implementation-screening pass: reject obviously stale code early, then spend human time on the properties that actually require statistical mechanics or MCMC expertise.

Next action

The colleague should inspect proposal symmetry, momentum refresh, Metropolis correction, and numerical stability. If those are part of the release claim, they need explicit fixtures and expected abstention cases.

If the team later wants the tool to discuss reversibility, add a dedicated derivation-audit example with an explicit verification boundary rather than stretching this term check beyond its contract.

Boundary

MathDevMCP does not certify an MCMC sampler. It shows that the code and document share selected structural ingredients and then stops before overclaiming ergodicity or correctness.

Chapter 23

Private Corpus Validation

Colleague scenario

The public fixtures are useful, but an industrial release also needs to prove that the machinery can validate private or sanitized department material without committing it. The private-corpus workflow is designed for that trust boundary. The manifest may contain sensitive local paths, while the report must remain safe to share.

Fixture and command

For this autonomous release, the evidence uses an external sanitized corpus created outside the repository. It includes six private-sanitized entries across DSGE, stochastic volatility, SDE/PDE numerics, ML objectives, ELBO/VI, and computational physics MCMC. A real deployment replaces that manifest with an approved private manifest using the same environment variable and validator.

MCP conversation

```
User:
  Validate the private corpus manifest, but do not reveal private paths in the
  answer.
Agent:
  I will validate the manifest and report only redacted release evidence.
Tool: validate_release_corpus(root, private_manifest="/secure/.../corpus.json")
Tool: release_readiness(root, profile="full")
Agent:
  The private manifest is loaded and release-gated entries validate. The
  normal report redacts private roots. If a path inside the git checkout were
  present, I would report it as a release finding rather than proceeding.
```

Listing 23.1: Private corpus validation summary

```
Command: MATHDEVMCP_PRIVATE_CORPUS_MANIFEST=<redacted-private-manifest>
    scripts/validate_private_corpus.sh "$PWD"
Status: consistent
Reason: Private corpus manifest is configured and satisfies release-
   gate privacy checks.
Private paths redacted: True
Findings: 0
Parser reports: [('private_dsge_macro_finance_euler', '
   selected_for_proof_audit'), ('
   private_stochastic_volatility_likelihood', 'selected_for_proof_audit
   '), ('private_sde_pde_numerics', 'selected_for_proof_audit'), ('
   private_ml_llm_objective', 'selected_for_proof_audit'), ('
   private_bayesian_elbo_vi', 'selected_for_proof_audit'), ('
   private_computational_physics_mcmc', 'selected_for_proof_audit')]
```

Output to inspect

The summary reports `consistent`, zero findings, redacted private paths, and parser reports selected for proof audit. The important part is not the temporary directory. The important part is that the validator used unredacted local paths internally while emitting redacted report output.

Interpretation

This closes the release blocker for private-corpus machinery without pretending that real confidential department documents were committed or inspected by this repository. It demonstrates that an operator can supply an external manifest and get release-gated parser evidence with safe output.

Next action

Before a department deployment, an operator should replace the sanitized corpus with an approved private manifest, verify coverage by domain, run `scripts/validate_private_corpus.sh`, and record whether the real private corpus covers the same domains as the sanitized release corpus.

That record matters for sales and governance. It lets a manager say, "The release machinery has been tested here; our department-specific claim begins only after we point it at our approved private material."

Boundary

This report's private evidence is sanitized external evidence. It does not claim that unredacted department-private documents were reviewed. The privacy rule is simple: populated private manifests stay outside git, normal reports redact paths, and any private path inside the checkout is a release finding.

The code owner for this boundary is `release_corpus.py`; the release profile owner is `release_policy.py`. Both should be reviewed before changing the private-corpus claim.

Part V

Security, Operations, and Maintenance

Chapter 24

Security and Privacy

Security in MathDevMCP starts from a local-first assumption: mathematical documents, private source trees, and code roots remain on the user's machine. The product does not need to upload documents to a service in order to search LaTeX labels, compare code terms, or build a release corpus report. This is a major part of the industrial value proposition for colleagues who work with unpublished notes, grant material, confidential model code, or department calibration files.

24.1 Private corpus boundary

Private documents must stay outside the repository. A populated private manifest may contain sensitive paths and should remain external. Normal MathDevMCP reports redact private document roots and code roots through `release_corpus.py`. Governance validation rejects private paths inside the checkout. Evidence bundles for review should be stored outside git unless they are deliberately reduced to safe report snippets.

The privacy contract has three levels. Public fixtures are safe to commit and are used for repeatable tests. External sanitized fixtures may be used to prove release machinery without revealing real documents. Real private manifests are operator-supplied and should never appear in git. The report uses the second level and says so plainly.

24.2 Command and backend governance

External commands are bounded by governance policy. `governance.py` checks that MathDevMCP subprocess calls include explicit timeouts, and the policy allowlist names the external tools the project expects: LaTeXML, Pandoc, Lean, Lake, and Sage. Backend failures are reported as structured evidence rather than raw shell surprises.

This matters for agent use. An MCP tool call should not become a hidden shell escape hatch. It should return a typed report with a status, findings, and provenance. When a

backend is unavailable, times out, or returns diagnostic-only evidence, the result should make that boundary visible.

24.3 Review checklist

Before sharing a release bundle, scan generated report evidence for absolute private paths, manifest filenames, and local checkout paths. Run `scripts/audit_release_report_substance.sh` and the release-readiness profile with the intended manifest. If a scan finds an unredacted private path, treat it as a release blocker, not a cosmetic documentation issue.

Chapter 25

Operations

Operations are intentionally command driven. A colleague should be able to clone the repository, install the package, run doctor checks, and obtain a release-readiness report without opening a notebook or editing the source. The normal operator sequence is:

```
PYTHONPATH=src python -m mathdevmcp.cli doctor
scripts/backend_env_doctor.sh "$PWD"
MATHDEVMCP_REQUIRE_LATEXML=1 scripts/validate_latexml_backend.sh
   "$PWD"
PYTHONPATH=src python -m mathdevmcp.cli benchmark-gate --root "
   $PWD"
PYTHONPATH=src python -m mathdevmcp.cli release-readiness --root
   "$PWD" --profile base
```

For full release validation, add:

```
export MATHDEVMCP_PRIVATE_CORPUS_MANIFEST=/secure/local/path/
   corpus.json
scripts/validate_private_corpus.sh "$PWD"
PYTHONPATH=src python -m mathdevmcp.cli release-readiness --root
   "$PWD" --profile full
```

25.1 Interpreting a run

Read the commands in layers. `doctor` describes local capability: Python, LaTeXML, Pandoc, Lean, Sage, SymPy, and LeanDojo. Backend env doctor checks the isolated environment used for LeanDojo. LaTeXML validation is strict only when the release profile requires it. The benchmark gate checks product behavior; release readiness combines those results with governance, parser policy, private-corpus status, and git state.

A dirty working tree is a caveat because release reports should be tied to a commit. It is normal during documentation editing, but it must be cleared or accepted explicitly

before a tag. Missing private corpus evidence is a caveat for the base profile and a blocker for the private-corpus and full profiles.

25.2 Environment layout

Lean may be available through `elan` in the user environment. LeanDojo is kept in `mathdevmcp-backends` and invoked through `MATHDEVMCP_BACKEND_CONDA_ENV`. This split is deliberate: it prevents the backend dependency stack from destabilizing the main environment while still making backend evidence available for strict profiles.

25.3 Release cadence

For a release branch, regenerate evidence after any change to parser policy, release corpus metadata, benchmark cases, release policy, or user-facing workflow examples. Rebuild the PDF after evidence regeneration. Run the substance audit before final review so page count cannot hide thin chapters.

Chapter 26

Maintainer Guide

Maintain release behavior by changing library code first, then CLI/MCP wrappers, then docs and evidence. When adding a new benchmark category, update the case builder, runner, summary tests, release report evidence, and release policy if the category becomes mandatory. When adding a new private corpus entry, keep the entry outside git and check redaction before sharing output.

The maintainer rule of thumb is simple: a release claim should have one owning module, one CLI or MCP surface, at least one test, and one report example if it is colleague-facing. If any of those four pieces is missing, the claim is not ready to sell as an industrial feature.

The highest-risk modules are:

- `benchmarks.py`, for benchmark totals and gate behavior;
- `release_policy.py`, for profile blockers and caveats;
- `release_corpus.py`, for privacy redaction and manifest shape;
- `parser_policy.py`, for proof-audit parser routing;
- `proof_audit_v2.py`, for verification-boundary reporting;
- `mcp_facade.py` and `cli.py`, for public tool surfaces.

26.1 Change workflow

For a behavior change, start with the library function and its direct tests. Then update the CLI command if arguments or JSON shape changed. Then update the MCP facade so agents receive the same behavior. Only then regenerate report evidence and edit narrative documentation. This order keeps prose downstream of executable behavior.

26.2 Contract hygiene

Do not rename statuses casually. Words such as `mismatch`, `unverified`, `inconclusive`, and `ready_with_caveats` are part of the user contract. A rename can break tests, release snippets, agent prompts, and colleague training material at the same time.

26.3 Documentation hygiene

Documentation changes that touch release claims should run the same gates as small code changes: generated evidence, report-substance audit, PDF build, path leak scan, and release-readiness check. The report is a sales document, but it is also a reproducibility artifact.

Chapter 27

Limitations and Accepted Boundaries

The product is conservative by design. It is useful because it knows when to stop. It does not convert every mathematical statement to a theorem prover. It does not guarantee semantic equivalence between arbitrary prose and code. It does not make private data safe to commit. It does not make numeric diagnostics into proofs. These boundaries should remain explicit in future releases.

27.1 Mathematical boundary

MathDevMCP can route obligations, expose missing assumptions, run diagnostic checks, and call deterministic backends when configured. It cannot turn a natural-language theorem, an incomplete proof sketch, or a nearby equation into a certified theorem by itself. If a result says `unverified` or `diagnostic_only`, an agent must preserve that language.

27.2 Parser boundary

Parser evidence is provenance evidence. It tells the user what label, file, environment, and line span were found. It does not guarantee that every macro, include, or semantic dependency has been understood. This is why parser policy and benchmark gates are release artifacts rather than implementation details.

27.3 Code-comparison boundary

Code–document comparison is a drift detector. A missing term can identify a review target, and a matched term can rule out some stale-code failures. Neither one is a proof of semantic equivalence. When a project uses domain-specific idioms such as factorizations,

automatic differentiation, or generated kernels, the benchmark should grow to recognize those idioms explicitly.

27.4 Operational boundary

Full release readiness depends on the operator's environment. LaTeXML, Lean, LeanDojo, and private-corpus manifests are external capabilities. MathDevMCP reports them; it does not pretend they exist when they do not. That honesty is part of the release claim.

Part VI

Detailed Release Manual

Chapter 28

End-to-End Colleague Scenario

This chapter walks through a realistic review day. A colleague has a document claim about a state-space likelihood and a Python implementation that is being edited quickly before a deadline. The colleague does not want a polished essay; they want to know where the claim lives, whether the code visibly contains the required numerical operations, whether a derivation can be certified, and what should be reviewed by a human.

28.1 Step 1: locate the candidate claim

The agent begins with `search-latex`. This avoids relying on a language model's memory of the repository. The search result gives labels and local context. The colleague can inspect those labels directly or ask the agent to use the top result. If no label is found, the workflow stops as *inconclusive* rather than inventing a source.

28.2 Step 2: extract neighborhood context

The label neighborhood gives surrounding prose and assumptions. This step matters because many mathematical claims are not self-contained. A likelihood equation may rely on notation introduced one paragraph above, while a stability condition may depend on a boundary condition or discretization convention.

28.3 Step 3: compare to code

The comparison step asks for concrete required terms such as `logdet`, `solve`, `gradient`, or `stability`. A missing term is not automatically a proof of a bug, but it is a precise review signal. The output includes the selected label, findings, and status, so the agent can explain what was checked.

28.4 Step 4: audit the derivation boundary

The v2 audit report is where MathDevMCP becomes most valuable for disciplined agent work. It records parser policy, typed diagnostics, shape diagnostics, numeric diagnostic suggestions, backend attempts, and a verification boundary. That boundary text is essential: it tells the colleague whether the output is a certificate, a diagnostic, or an abstention.

28.5 Step 5: produce an implementation brief

For handoff, the implementation brief bundles the key evidence. A teammate can read one object rather than rerun every command. This is especially useful when the agent is asked to create a code-review note, a pull-request comment, or a maintenance ticket.

Chapter 29

Release Gate Interpretation

The release gate should be read as a contract, not a vibe check. A green base profile means public fixtures, governance, release corpus metadata, parser policy, and benchmark cases pass. It does not mean every optional backend or private corpus is available. A green full profile means the strict optional evidence has also been supplied. This separation lets the team use MathDevMCP productively on ordinary machines while still preserving a stricter bar for industrial release.

29.1 Blockers

Blockers represent release conditions that must be fixed or explicitly scoped out. Examples include benchmark failure, parser policy failure, governance validation failure, missing strict LaTeXML evidence in the LaTeXML profile, missing backend evidence in the backend profile, and missing private manifest evidence in the private-corpus or full profiles.

29.2 Caveats

Caveats are not ignored. They are tracked because a release may be useful while still carrying environmental or evidence limitations. A dirty worktree is a caveat because generated reports should name the actual commit. Missing private corpus evidence is a low-severity caveat in the base profile, but a blocker in the private-corpus and full profiles.

29.3 Expected abstentions

Expected abstentions protect the product from false confidence. A benchmark can pass because the tool correctly abstained in a case where certification would be inappropriate. This is different from a tolerated failure budget. The current benchmark policy requires all cases to pass.

Chapter 30

Private Corpus Operating Model

Private corpus validation is designed for teams that cannot commit research documents, code paths, or real local directory structures. The populated manifest lives outside git. The release validator reads it locally, uses unredacted paths internally to parse documents, and emits redacted reports.

30.1 External sanitized corpus

For autonomous release validation in this workspace, the execution created an external sanitized corpus under a temporary directory. It includes six release domains: DSGE macro-finance, stochastic volatility, SDE/PDE numerics, ML/LLM objectives, Bayesian ELBO/VI, and computational-physics MCMC. The generated manifest is not committed. The report includes only redacted summaries.

30.2 Real private corpus substitution

A real deployment should replace the sanitized manifest with an approved external manifest. The same validator and full profile commands apply. The operator should verify that every release-gated entry has expected labels, parser backends, expected abstentions or false-confidence seeds, and external document roots. If a real private corpus covers fewer domains than the sanitized release corpus, the release report should say exactly that.

30.3 Failure modes

Important private-corpus failure modes are:

- missing manifest path;

- invalid JSON;
- malformed entry fields;
- private path inside the repository checkout;
- missing document root or code root;
- release-gated entry with no expected labels;
- release-gated entry with no parser backend;
- parser policy not selected for proof audit.

Chapter 31

Backend Environment Operating Model

Backend tools are intentionally not all installed into the main Python environment. This is a release-quality design choice. LeanDojo can bring a dependency stack that conflicts with unrelated document or ML packages. The backend env checks therefore prove that the optional backend can be invoked without requiring the main working environment to import every optional module.

31.1 Lean

Lean is detected as an executable, usually through the user-local `elan` proxy. The release pins the intended toolchain through `MATHDEVMCP_LEAN_TOOLCHAIN` for MathDevMCP subprocesses. This avoids changing the user's global Lean default.

31.2 LeanDojo

LeanDojo is checked as a Python module inside `mathdevmcp-backends`. A plain `doctor` command in the active environment may report LeanDojo as unavailable. The backend env doctor is the authoritative check for the backend profile.

31.3 LaTeXML

LaTeXML is a system executable, not a Python dependency. The strict LaTeXML profile requires it to preserve expected labels on the benchmark corpus. The current evidence shows label preservation and source provenance. Environment recognition differs from the lightweight parser, so parser policy still chooses the current parser for proof-audit routing.

Chapter 32

Code Architecture for Maintainers

The implementation should be maintained by preserving the boundary between library logic, CLI rendering, MCP wrappers, scripts, and documentation.

32.1 Library logic

Library functions build structured dictionaries with contract metadata. They should be deterministic for the same input files and environment. New behavior should be added in library code first and exercised by direct tests.

32.2 CLI

The CLI should remain a thin argument parser. It should call library functions, print JSON, and return meaningful exit codes. It should not contain hidden business logic that bypasses the tested library surface.

32.3 MCP facade

The MCP facade should wrap the same library functions. Tool names are part of the colleague-facing API. If a tool is renamed, keep a compatibility alias or document a breaking change.

32.4 Scripts

Scripts should orchestrate repeatable workflows. They should set `PYTHONPATH` explicitly, avoid shell string evaluation for backend commands, and preserve redaction. Scripts that create generated evidence should write to ignored or explicitly requested output directories.

Chapter 33

Adding a New Domain

To add a new mathematical domain, follow this release path:

1. Add a small public or sanitized fixture with at least one labeled mathematical block.
2. Add paired code showing the relevant operations or a seeded mismatch.
3. Extend AST operation extraction only if the new operation is genuinely needed.
4. Add benchmark cases for parser preservation, AST operation coverage, and expected abstention behavior.
5. Add release corpus metadata.
6. Add a report case study and generated evidence snippet if the domain is release-facing.
7. Update private manifest templates only with placeholder paths.

This path keeps new domains measurable. It also prevents the product from accumulating impressive-sounding workflows that do not have regression tests.

Chapter 34

Adding a New Backend

A new backend should enter the system as optional evidence. The correct path is to add a doctor capability, an isolated invocation strategy if needed, timeout protection, a clear result contract, and tests for unavailable, timeout, inconclusive, mismatch, and certifying behavior. Only after that should the backend be used in proof-audit routing or release profiles.

34.1 Backend result discipline

Backend results should distinguish:

- backend unavailable;
- input not encodable;
- timeout;
- diagnostic-only evidence;
- counterexample or mismatch;
- deterministic certificate.

The last item is intentionally rare. Most mathematical development tools are useful because they reduce search space and expose gaps, not because they automatically certify every claim.

Chapter 35

Release Evidence Maintenance

The evidence snippets in this report are generated by `scripts/generate_release_report_evidence.sh`. Regenerate them after changing release policy, parser policy, benchmark cases, private corpus validation, or user-facing workflow examples. Do not hand-edit generated evidence unless the file is explicitly converted into narrative documentation.

35.1 Evidence hygiene

After regeneration, run a path leak scan. The scan should check for real private roots, local manifest filenames, and repository-local absolute paths. The committed snippets may contain `<repo>`, `<redacted-private-path>`, and `<redacted-private-manifest>`.

35.2 Evidence reproducibility

Generated snippets should name the command that produced them. A colleague reading the report should be able to rerun the command in the repository and understand why the local result may differ if the worktree is dirty, optional backends are missing, or a private manifest is not configured.

Chapter 36

Risk Register

36.1 Risk: overclaiming certification

The most important product risk is false confidence. Mitigation: keep verification-boundary text in proof-audit reports, preserve diagnostic-only flags, and require benchmark cases where abstention is the correct outcome.

36.2 Risk: private data leakage

Private roots and manifests are sensitive. Mitigation: keep populated manifests outside git, redact normal reports, reject private paths inside the checkout, and scan generated docs before commit.

36.3 Risk: backend dependency conflict

Optional backends can conflict with the main environment. Mitigation: use `mathdevmcp-backends`, backend-specific doctor checks, and explicit environment variables rather than broad installation into the base env.

36.4 Risk: benchmark drift

Benchmark totals and category meanings can drift as the project grows. Mitigation: update tests, reset memo, and release report together whenever a benchmark case is added or removed.

36.5 Risk: large-module entropy

The codebase has several large modules. Mitigation: keep public facades stable, extract pure helper modules when it improves locality, and run focused tests after each extraction.

Appendices

Appendix A

Release Corpus Summary

Listing A.1: Release corpus validation summary

```
Command: PYTHONPATH=src python -m mathdevmcp.cli validate-release-
    corpus --root "$PWD/benchmarks/fixtures"
Status: consistent
Findings: 0
Entries: 22
Private manifest status: loaded
Private paths redacted: True
```

Appendix B

Machine-Readable Evidence Excerpts

The full generated JSON excerpts remain committed under `docs/generated/release_report/`. They are intentionally referenced here rather than printed in full so that the report foregrounds colleague workflows and MCP conversations while preserving reproducibility.

Evidence file	Purpose
`benchmark-gate-json-excerpt.txt`	CI-friendly benchmark gate status, category totals, and expected abstentions.
`parser-policy-json-excerpt.txt`	Parser role decision and proof-audit routing boundary.
`release-corpus-json-excerpt.txt`	Public and configured private corpus manifest summary with redaction metadata.
`release-readiness-full-json-excerpt.txt`	Full profile blockers, caveats, capability evidence, and profile requirements.
`private-corpus-json-excerpt.txt`	Private-corpus validation payload with private paths redacted.

Appendix C

Evidence Index

Listing C.1: Generated evidence index

```
Generated release report evidence snippets:
doctor-summary.txt
benchmark-gate-summary.txt
parser-policy-summary.txt
release-corpus-summary.txt
release-readiness-full-summary.txt
private-corpus-summary.txt
workflow-search.txt
workflow-context.txt
workflow-compare.txt
workflow-audit.txt
workflow-brief.txt
case-kalman-search.txt
case-kalman-compare.txt
case-kalman-brief.txt
case-hmc-search.txt
case-hmc-compare.txt
case-hmc-brief.txt
case-macro-filter-search.txt
case-macro-filter-compare.txt
case-macro-filter-brief.txt
case-dsge-search.txt
case-dsge-compare.txt
case-dsge-brief.txt
case-stochastic-volatility-search.txt
case-stochastic-volatility-compare.txt
case-stochastic-volatility-brief.txt
case-sde-pde-search.txt
case-sde-pde-compare.txt
case-sde-pde-brief.txt
case-ml-objective-search.txt
case-ml-objective-compare.txt
```

```
case-ml-objective-brief.txt
case-elbo-search.txt
case-elbo-compare.txt
case-elbo-brief.txt
case-physics-mcmc-search.txt
case-physics-mcmc-compare.txt
case-physics-mcmc-brief.txt
benchmark-gate-json-excerpt.txt
parser-policy-json-excerpt.txt
release-corpus-json-excerpt.txt
release-readiness-full-json-excerpt.txt
private-corpus-json-excerpt.txt
```

Appendix D

Command Reference

Command	Purpose
`doctor`	Report runtime capabilities and optional backend status.
`search-latex`	Search a LaTeX source tree for relevant context.
`extract-latex-neighborhood`	Extract paragraph context around a label.
`compare-label-code`	Compare labeled document terms to code.
`audit-derivation-v2-label`	Build typed proof-audit evidence for a label.
`implementation-brief`	Bundle search, context, and code checks.
`benchmark-gate`	Run the release benchmark gate.
`validate-release-corpus`	Validate public and configured private corpus metadata.
`release-readiness`	Produce a profile-specific release report.

Appendix E

Private Manifest Fields

Field	Meaning
`id`	Stable entry identifier.
`domain`	Mathematical or colleague workflow domain.
`privacy_class`	`private_external` or `private_sanitized_external`.
`document_root`	External LaTeX/document root.
`code_roots`	External code roots associated with the documents.
`expected_labels`	Labels the parser must preserve.
`expected_operations`	Mathematical operations expected in code or AST evidence.
`expected_abstentions`	Known cases where conservative abstention is expected.
`seeded_false_confidence_cases`	False-confidence seeds for review.
`required_parser_backends`	Parser backends required for the entry.
`release_gate_enabled`	Whether the entry is mandatory for private profile evidence.
`notes`	Human-readable release note.